

[← Back to CCDV-F Sample Tests](#)

Great Job!

Tools and MCPs Practice Test Complete

10/10

Correct

100%

Score

7:05

Time



This was a sample test from a limited pool of questions.
Upgrade to Premium for the complete CCDV-F bank with hundreds more questions!

Upgrade to Premium

Get unlimited access to:

- ✓ Unlimited practice tests across all domains
- ✓ Full-length 53-question mock exams with realistic timing
- ✓ Detailed explanations and study guides
- ✓ Progress tracking and performance analytics
- ✓ Smart practice focusing on weak areas

Review Your Answers

1



→ Login to Clarify

A developer adds a customer lookup tool to a Claude-powered support app. In testing, Claude emits

the following tool request, but no customer data appears in the conversation.

Artifact:

```
assistant tool_request:
  name: lookup_customer
  arguments: { "email": "sam@example.com" }
app log:
  received assistant message
  no tool runner configured for lookup_customer
  returned final text to user
```

What is the BEST next implementation step?

- A. Add a harness step that validates the arguments, calls `lookup_customer`, and returns the tool result to Claude.



- B. Rename the tool to a shorter name so Claude can execute the customer lookup directly.

- C. Move the customer API key into the prompt so Claude can complete the lookup itself.

- D. Ask Claude to avoid tool calls and infer customer details from the conversation context.

Explanation:

Correct answer (A): Claude emitting a tool request does not execute the external action. The surrounding application or agentic harness must receive the tool call, validate its arguments against the schema, dispatch the mapped function or API request, and provide the tool result back to Claude so it can continue the task.

Why the other options are wrong:

- Option B: A clearer tool name can improve selection, but the trace shows the tool was already selected and no runner executed it.
- Option C: Putting credentials in the prompt is not an execution mechanism and would create unnecessary credential exposure risk.
- Option D: Inferring customer data from context avoids the required external lookup and would not produce reliable account information.

2



→ Login to Clarify

Claude is frequently choosing the wrong internal support tool. The current tool definitions are shown below.

Artifact:

```
Tool 1
name: get_info
description: Gets information.
input_schema: { "type": "object", "properties": { "id": { "type": "string" } } }

Tool 2
name: fetch_data
description: Fetches data from systems.
input_schema: { "type": "object", "properties": { "id": { "type": "string" } } }
```

The app needs one tool for customer profile lookup and another for order shipment lookup. What is the BEST primary fix?

A.

Give each tool a specific name, distinct usage description, typed parameters, and required fields.



B.

Keep the schema unchanged and add a system prompt telling Claude to be more careful.

C.

Merge both tools into one generic tool that decides the lookup type at runtime.

D.

Remove parameter schemas so Claude can send any arguments it thinks are useful.

Explanation:

Correct answer (A): Reliable tool selection depends on clear, distinguishable tool definitions. Specific names, precise descriptions, explicit parameter types, required fields, and constraints help Claude decide when to use each capability and call it with valid inputs.

Why the other options are wrong:

- Option B: Prompt guidance may help slightly, but it does not fix the vague and overlapping tool definitions that are causing confusion.
- Option C: A generic combined tool can hide distinctions and push ambiguity into runtime dispatch rather than helping Claude choose correctly.
- Option D: Removing schemas reduces validation and makes malformed or ambiguous tool calls more likely.

3

[→ Login to Clarify](#)

A support assistant has these tools available in production.

Artifact:

Tool: `get_ticket_status`

Effect: read-only lookup

Visible outside app: no

Tool: `close_ticket`

Effect: changes ticket state and emails customer

Visible outside app: yes

The product requirement is to keep status checks fast while preventing accidental customer-visible actions. Which execution pattern is MOST appropriate?

- A.
Run `get_ticket_status` automatically, but require explicit approval before `close_ticket` executes.



- B.
Require human approval for both tools every time Claude requests either one.

- C.
Allow both tools to execute automatically after Claude selects the right tool.

- D.
Disable tool use and ask Claude to draft instructions for a human operator.

Explanation:

Correct answer (A): Read-only lookups and state-changing actions should have different execution controls. A read-only status check can be automated for speed, while a customer-visible state change should require approval or confirmation before the harness executes it.

Why the other options are wrong:

- Option B: Approval for every read-only lookup would reduce the required speed without adding much value for the stated low-risk action.

- Option C: Automatic execution for the state-changing email-producing tool does not address accidental customer-visible actions.
- Option D: Disabling all tools avoids automation entirely and fails the requirement to keep status checks fast.

4

[→ Login to Clarify](#)

A team is designing an MCP server for a billing assistant.

Artifact:

Needed capability: `submit_refund(case_id, amount)`

Behavior: calls the billing API and creates a refund transaction

Also available: refund policy document and refund explanation template

How should `submit_refund(case_id, amount)` be exposed through MCP?

A.

As an MCP tool, because it performs an executable external action.



B.

As an MCP resource, because it relates to billing data.

C.

As an MCP prompt, because it helps Claude discuss refunds.

D.

As plain context only, because MCP should not call APIs.

Explanation:

Correct answer (A): An MCP tool represents an executable capability such as querying or updating an external system. Because ``submit_refund`` calls a billing API and creates a transaction, it should be exposed as an MCP tool, while the policy document would fit a resource and the explanation template would fit a prompt.

Why the other options are wrong:

- Option B: An MCP resource is appropriate for readable data or context, not for executing a refund transaction.
- Option C: An MCP prompt is a reusable template or interaction pattern, not the API action that creates the refund.
- Option D: MCP can expose executable tools; the issue is to classify the API action correctly and control execution appropriately.

5

[→\] Login to Clarify](#)

A platform team maintains a Jira integration for Claude-powered workflows.

Artifact:

```
Clients needing the same integration:
- internal web support assistant
- desktop research assistant
- IDE-based engineering assistant
Capabilities: search_issues, create_issue, update_issue_status
Goal: avoid duplicating integration code and permission logic
```

Which architecture is the BEST fit?

A.

Expose the Jira capabilities through a shared MCP server used by the clients.



B.

Copy the same custom tool code into each client and update them separately.

C.

Replace the integration with a Skill that explains Jira workflows to Claude.

D.

Put Jira REST examples in each prompt and let Claude construct requests.

Explanation:

Correct answer (A): MCP is well suited when the same organization-specific tools, resources, or prompts should be reused by multiple Claude-compatible clients. A shared MCP server centralizes the Jira integration and permission logic instead of duplicating it across applications.

Why the other options are wrong:

- Option B: Duplicating custom tool code may work initially but conflicts with the stated maintainability goal across multiple clients.
- Option C: A Skill can package instructions, but it is not a replacement for centralized executable Jira API capabilities.
- Option D: Prompt examples do not provide controlled dispatch, validation, credential handling, or reusable integration logic.



A developer is integrating a local MCP server into a desktop Claude client.

Artifact:

Deployment: runs only on the user's laptop

Startup: client launches the server as a child process

Network access: not required

Communication pattern: local process streams

Which transport choice is MOST appropriate?

A.

Use stdio transport between the client and the launched local server process.



B.

Use a network socket transport because all MCP servers must be remote services.

C.

Avoid MCP and paste the local files into every conversation manually.

D.

Use an MCP prompt instead of transport because prompts can carry tool calls.

Explanation:

Correct answer (A): stdio-based MCP communication is appropriate when the client launches or communicates with a local server process. Socket-based transport is more appropriate for a network-accessible service or separate client/server deployment over a network.

Why the other options are wrong:

- Option B: Socket transport can be appropriate for networked services, but the artifact specifies a local child process with no network requirement.

- Option C: Manual pasting is not a reusable integration and does not expose executable MCP capabilities.
- Option D: An MCP prompt is a reusable template, not the communication transport for a server process.

7

[→ Login to Clarify](#)

A Claude app fails when trying to fetch subscription details. The trace is shown below.

Artifact:

```
Tool schema:
name: get_subscription
required: ["customer_id"]
properties:
  customer_id: string

Claude tool request:
name: get_subscription
arguments: { "email": "lee@example.com" }

Harness result:
validation_error: missing required field customer_id
```

What is the MOST likely root cause?

A.

Claude selected the tool but supplied arguments that do not match the declared schema.



B.

The external subscription API is down because the harness reached validation.

C.

The model tier is too small to use tools in a production application.

D.

The MCP transport is wrong because validation errors only occur over sockets.

Explanation:

Correct answer (A): The trace shows the tool name matches, but the arguments fail schema validation because ``customer_id`` is required and only ``email`` was supplied. Tool failures can originate from schema or argument mismatches before any external API call occurs.

Why the other options are wrong:

- Option B: The harness stopped at validation, so the evidence does not show that the external API was called or unavailable.
- Option C: The trace points to a concrete argument/schema mismatch, not a general model capability issue.
- Option D: The artifact does not describe MCP transport, and validation errors are not specific to socket-based communication.

8

[→ Login to Clarify](#)

A sales assistant uses a CRM search tool. Claude only needs to recommend the top matching account, but the tool currently returns the full CRM payload.

Artifact:

```
crm_search result:
- 500 account objects
- full notes history for each account
- internal audit metadata
```

- raw API pagination fields

Observed effect: responses are slower and Claude focuses on irrelevant notes

What is the BEST change to the tool result returned to Claude?

- A.
Return a concise structured summary with the top matches and fields needed for the next step.



- B.
Return the same payload but ask Claude to ignore irrelevant fields in the prompt.

- C.
Return all CRM fields as plain text so Claude can decide what matters later.

- D.
Disable the CRM tool and require the user to provide account details manually.

Explanation:

Correct answer (A): Tool results should be concise, relevant, and structured enough for Claude to continue the task. Filtering to the top matches and needed fields reduces context bloat and decreases the chance that irrelevant data distracts the model.

Why the other options are wrong:

- Option B: Prompt instructions do not remove unnecessary tokens or prevent irrelevant payload fields from entering context.
- Option C: Converting all fields to plain text worsens structure and still includes excessive irrelevant information.
- Option D: Manual data entry avoids the tool rather than fixing the result-shaping problem in the integration.

9

[→\] Login to Clarify](#)

A developer adds a tool definition to a Claude-powered ticketing assistant, but no ticket is created even though Claude returns a tool request.

Artifact:

```
assistant output:
type: tool_use
name: create_ticket
input: { title: 'VPN issue', priority: 'high' }
server log:
'Received assistant response; sent text back to user. No tool dispatcher configured.'
```

What is the best next implementation step?

A.

Add application-side dispatch code that validates the input, calls the ticket API, and returns the tool result.



B.

Rename the tool to a more descriptive name so Claude can create the ticket without server code.

C.

Put the ticket API endpoint in the system prompt so Claude can invoke the endpoint directly.

D.

Convert the tool request into plain text and ask the user to submit the ticket manually.

Explanation:

Correct answer (A): Claude can select or request a tool call, but the developer's application, agent harness, or MCP server is responsible for validating the arguments, executing the external operation, and returning the result. The trace shows that Claude produced a valid-looking tool request, but the server only relayed the response to the user and had no dispatcher. Implementing the dispatcher is the direct fix.

Why the other options are wrong:

- Option B: A clearer name can help tool selection, but the trace shows the tool was already selected. The missing piece is execution by the application or server.
- Option C: Putting an endpoint in a prompt does not cause Claude to execute HTTP requests. External calls still require implemented tool dispatch or an MCP server.
- Option D: Manual submission avoids the integration but does not address the intended tool-use architecture for creating tickets from the assistant.

10 

[→ Login to Clarify](#)

A project-management assistant often sends invalid status values to a downstream API. The API accepts only ``todo`` , ``in_progress`` , and ``done`` .

Artifact:

```
name: update_task
description: Updates a task.
input_schema:
  type: object
  properties:
    taskId: string
    status: string, description: New status
  required: taskId, status
```

Which change best addresses the recurring invalid status problem?

A.

Add an enum constraint for ``status`` and describe when each allowed value should be used.



B.

Remove ``status`` from required fields so the API can infer the intended value.

C.

Rename ``update_task`` to ``update_project_task_status_now`` without changing the schema.

D.

Keep the schema unchanged and ask Claude in the prompt to avoid invalid statuses.

Explanation:

Correct answer (A): When downstream systems require known values, the tool schema should constrain the input with an enum and clear field descriptions. This gives Claude and the application a validation-friendly contract and reduces ambiguous free-text arguments. The issue is not merely the tool name or prompt wording; the schema currently permits any string.

Why the other options are wrong:

- Option B: Making ``status`` optional avoids validation at the schema level and does not help the API receive one of its required valid values.
- Option C: A more specific name may help tool selection, but the failure is invalid argument content, not confusion about which tool to call.
- Option D: Prompt instructions can help, but leaving the schema unconstrained continues to permit invalid free-text values.